

TECHNOLOGIES IN EDUCATION
UNIVERSITY NSU

MICROELECTRONICS
INNOVATIONS
CATALYTIC
MATERIALS
ASSEMBLY
POINT **DRUG**
DESIGN

SCIENTIFIC
LABORATORY
HYBRID
MATERIALS
GEOPHYSICS
ENGINEERING
ENERGY CONSERVATION
BIOTECHNOLOGY
GEOCHEMISTRY
NANOTECHNOLOGY

HIGH
ENERGIES
SEMIOTICS
SCIENCE
MATHEMATICAL MODELING

IT
DEEP
LEARNING
BRAIN
STUDY
COGNITIVE

DEVELOPMENT
ELEMENTARY
PARTICLES
THE ARCTIC REGIONS
DARK
MATTER

QUANTUM
TECHNOLOGIES
BIOMEDICINE
APPLIED
STUDIES
PHOTONICS
ASTRONOMY
GLOBAL PRIORITY
ASTROPHYSICS
BIOINFORMATICS

LASER
PHYSICS
KNOWLEDGE
ECONOMY

GEOLOGY
ARCHEOLOGY
TECHNOLOGIES

N* Novosibirsk
State
University
*THE REAL SCIENCE

Машинное обучение

Семинар 12

Глушенко Андрей Валерьевич
Институт Интеллектуальной
Робототехники



**FASTAPI
PERFORMANCE**

ME

FLASK

Введение

Развертывание модели машинного обучения — важный шаг на пути к тому, чтобы ваши модели стали доступны пользователям и приложениям.

В основном, развертывание модели машинного обучения можно разделить на два этапа:

1. Разработка приложения для доступа к моделям для прогнозирования и использования.
2. Развертывание приложения в тестовой, промежуточной и производственной средах.

Мы рассмотрим некоторые из лучших и популярных инструментов для этой цели, **FastAPI** (и постараемся **Docker**).

FastAPI — это фреймворк, обеспечивающий высокую производительность и простоту использования при разработке веб-приложений и **API**.

Docker — это платформа контейнеризации, которая позволяет упаковывать ваше приложение и его зависимости в контейнеры.

Развёртывание модели. Жизненный цикл

- 1. Разработка модели:** На этом этапе разрабатывается, обучается и тестируется модель машинного обучения на основе предоставленных данных (с которыми мы уже знакомы).
- 2. Создание API:** Разработка веб-приложения/автономного программного обеспечения, предоставляющего доступ к модели.
- 3. Контейнеризация:** После создания приложения, оно и его зависимости упаковываются в контейнер **Docker**, обеспечивая изолированное и переносимое выполнение.
- 4. Тестирование и валидация:** В контексте **CI/CD Docker Compose** можно использовать для локального тестирования и валидации приложения в изолированной среде перед его развёртыванием.
- 5. Развёртывание приложения:** Подготовленный контейнер приложения развёртывается на выбранной платформе или инфраструктуре. Это может быть любая платформа оркестровки контейнеров или **Kubernetes**.
- 6. Оркестрация и управление:** В конце используется инструмент для управления жизненным циклом контейнера и обеспечения высокой доступности.

Асинхронное программирование

- **Асинхронная архитектура** — это неблокирующая архитектура, что означает, что она не блокирует дальнейшее выполнение, пока выполняется одна или несколько операций.
- В терминологии программирования **асинхронное программирование** — это метод, позволяющий вашей программе запускать потенциально длительную задачу и при этом оставаться отзывчивой к другим событиям во время выполнения этой задачи, вместо того чтобы ждать ее завершения.

Преимущества асинхронного кода

- Асинхронные запросы позволяют обрабатывать несколько запросов одновременно без блокировки, что может повысить пропускную способность вашего приложения.
- Асинхронный код обеспечивает более эффективное использование ресурсов сервера, поскольку он не блокирует поток, ожидающий завершения операций ввода-вывода.
- Асинхронный код может повысить скорость отклика вашего приложения, поскольку запросы могут обрабатываться независимо друг от друга, избегая ожидания и ускоряя обработку.
- Многоядерные процессоры повышают производительность за счет одновременного выполнения нескольких операций, что особенно полезно для параллельных задач. Асинхронный код, в свою очередь, повышает эффективность выполнения задач в однопоточной среде, освобождая основной поток выполнения.

Где не использовать асинхронный код

- Предположим, мы вычисляем произведение двух огромных матриц. Такие операции сильно зависят от процессора, поскольку требуют значительных вычислительных ресурсов и выполняются на процессоре. В этом случае использование асинхронного кода не даст преимущества, так как нет возможности передать управление другим задачам до завершения текущей задачи. Вместо этого, чтобы ускорить вычисления, следует использовать параллельные потоки или процессы для распределения нагрузки между несколькими процессорными кодами.
- В Python, например, можно использовать модуль `multiprocessing` для создания процессов, которые могут работать параллельно и обрабатывать различные части произведения матриц одновременно. Это позволит более эффективно использовать вычислительные ресурсы и сократить общее время вычислений.

Примеры использования асинхронного режима в сервисах машинного обучения с FastAPI

- Асинхронный эндпоинт для ресурсоемких вычислений. Для вычислительных задач, требующих больших вычислительных ресурсов ЦП, асинхронное программирование не обеспечит повышения производительности. Однако, если объединить такие задачи с операциями ввода-вывода, это может помочь оптимизировать весь процесс. Например, можно выполнять вывод в отдельном потоке, чтобы не блокировать основной поток выполнения.
- Асинхронный эндпоинт для запросов к базе данных. Если вашей модели требуются данные из базы данных для вывода, асинхронный доступ к базе данных может повысить производительность.
- Асинхронный эндпоинт для внешних API. Если ваша модель зависит от данных внешнего API, асинхронные запросы могут помочь избежать блокировки выполнения.

FastAPI

- **FastAPI** — это современный веб-фреймворк на Python, основанный на асинхронном программировании.
- **Преимущества FastAPI:**
 - **Простота использования:** **FastAPI** предоставляет простой и понятный способ создания **API** с использованием стандартных методов и синтаксиса **Python**, таких как аннотации типов данных и декораторы маршрутизации.
 - **Производительность:** **FastAPI** обеспечивает высокую производительность, в том числе за счет использования асинхронных запросов.
- **Недостатки FastAPI:**
 - Поскольку **FastAPI** — относительно новый фреймворк, он имеет меньше сторонних библиотек и дополнений, чем более известные веб-фреймворки на **Python**, такие как **Flask** или **Django**.

Основные объекты и функции FastAPI

- Маршруты (Routes)
- APIRouter
- Методы HTTP запросов
- Модели **Pydantic**
- Ответы (Responses)
- Фооновые задачи (Background Tasks)
- Работа с файлами
- Lifespan(Startup/Shutdowns Events)
- Middleware

Практические аспекты

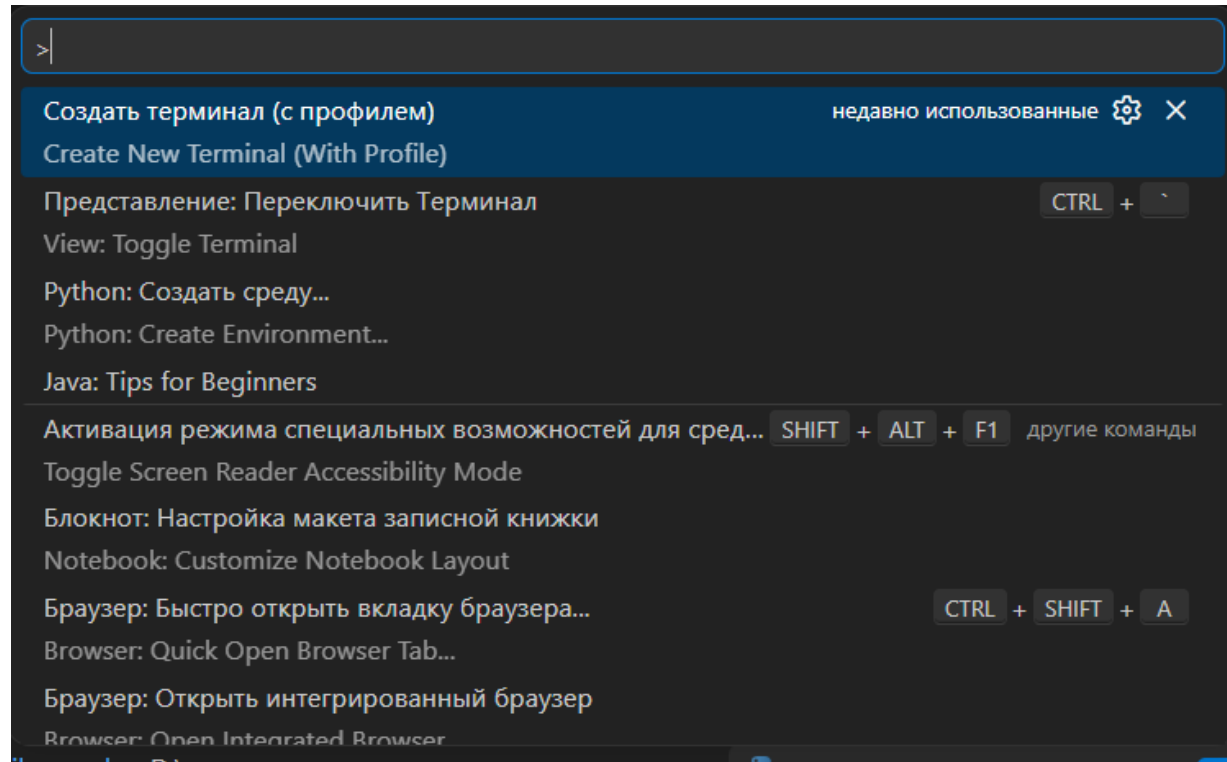
Создание .venv

Моё первое приложение на FastAPI

10

Создание .venv в VSCode

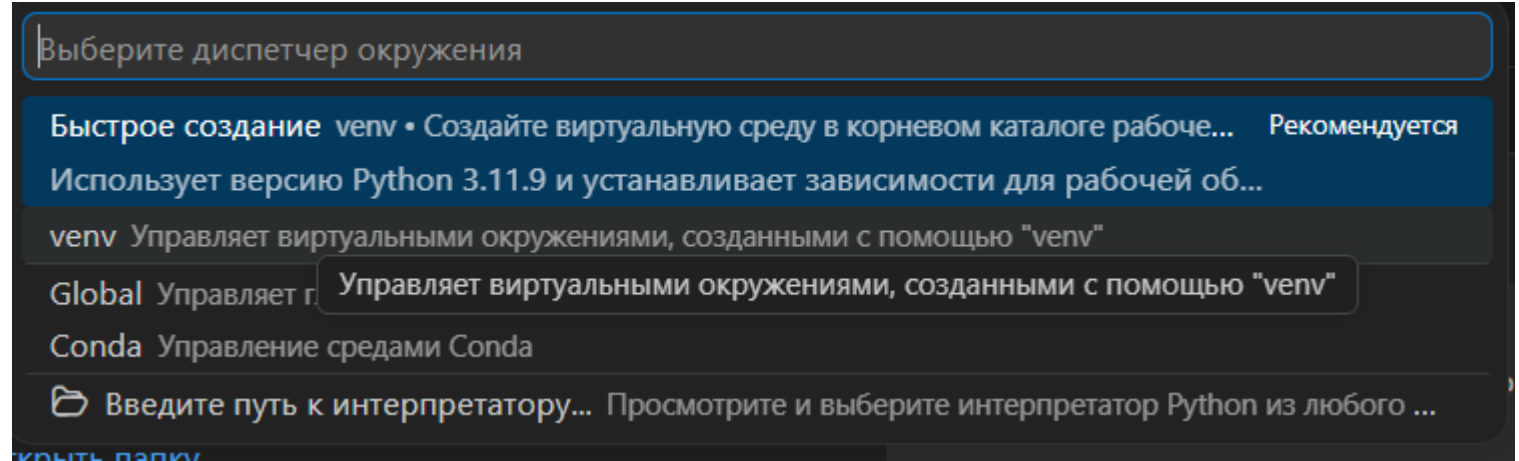
- Откройте VSCode и нажмите Ctrl+Shift+P (винда)



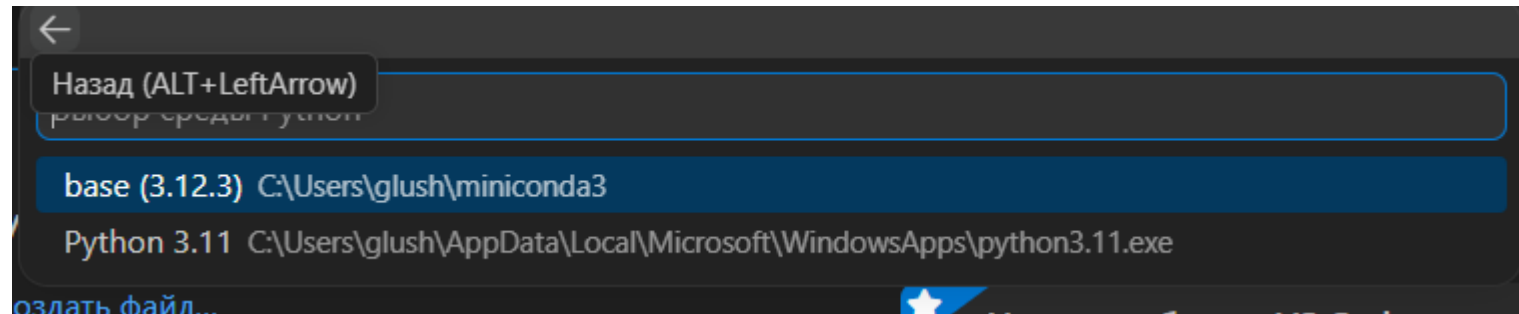
- Выберите «Python: Создать среду...»

Создание .venv в VSCode

- Выберите «venv»

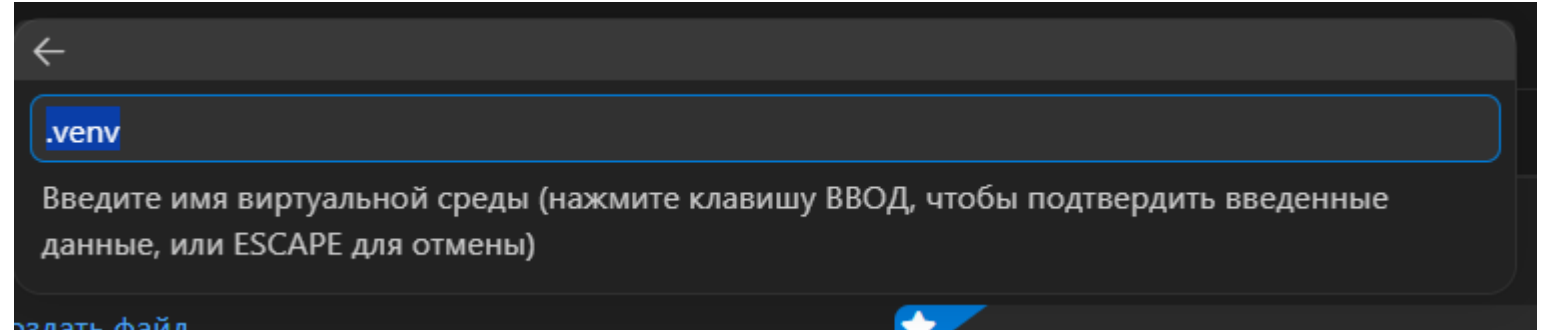


- Выберите своё окружение Python

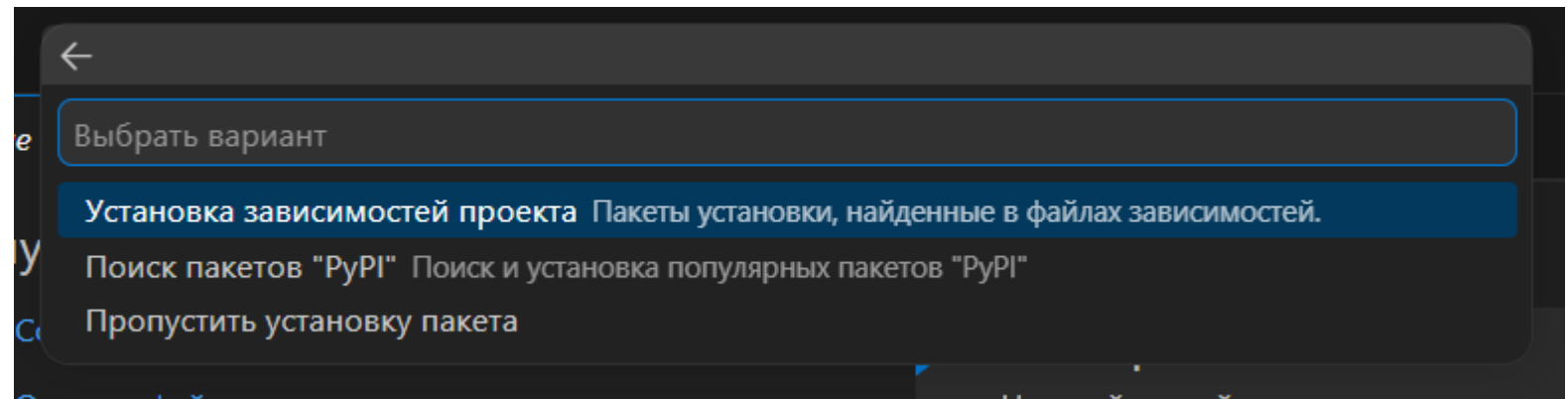


Создание .venv в VSCode

- Нажмите Enter

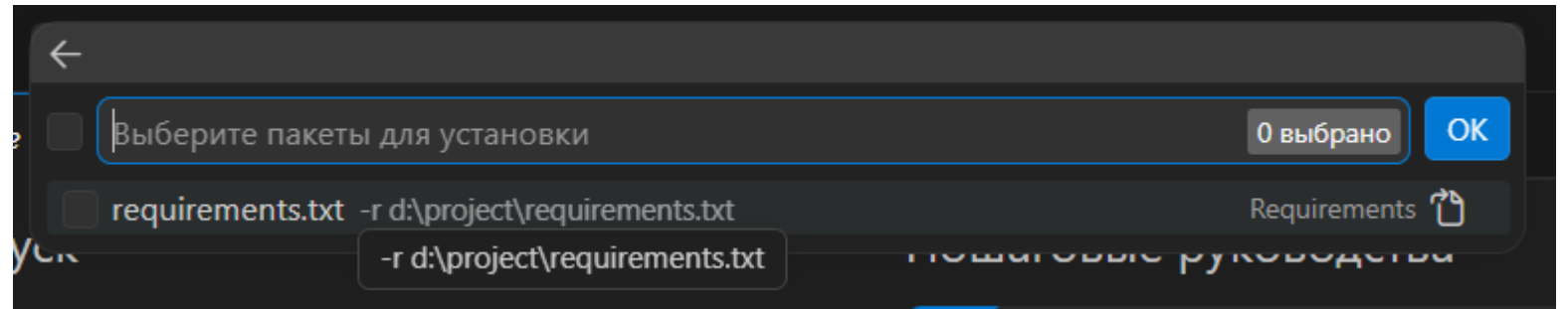


- Выберите «Установка зависимостей проекта...»



Создание .venv в VSCode

- Выберите файл requirements.txt, куда предварительно добавили необходимые библиотеки



- Пример файла requirements.txt для работы с FastAPI

```
requirements.txt X
requirements.txt
1 fastapi[standard]
```

Пример простого приложения на FastAPI

- Создадим файл `main.py` с таким содержимым:

```
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

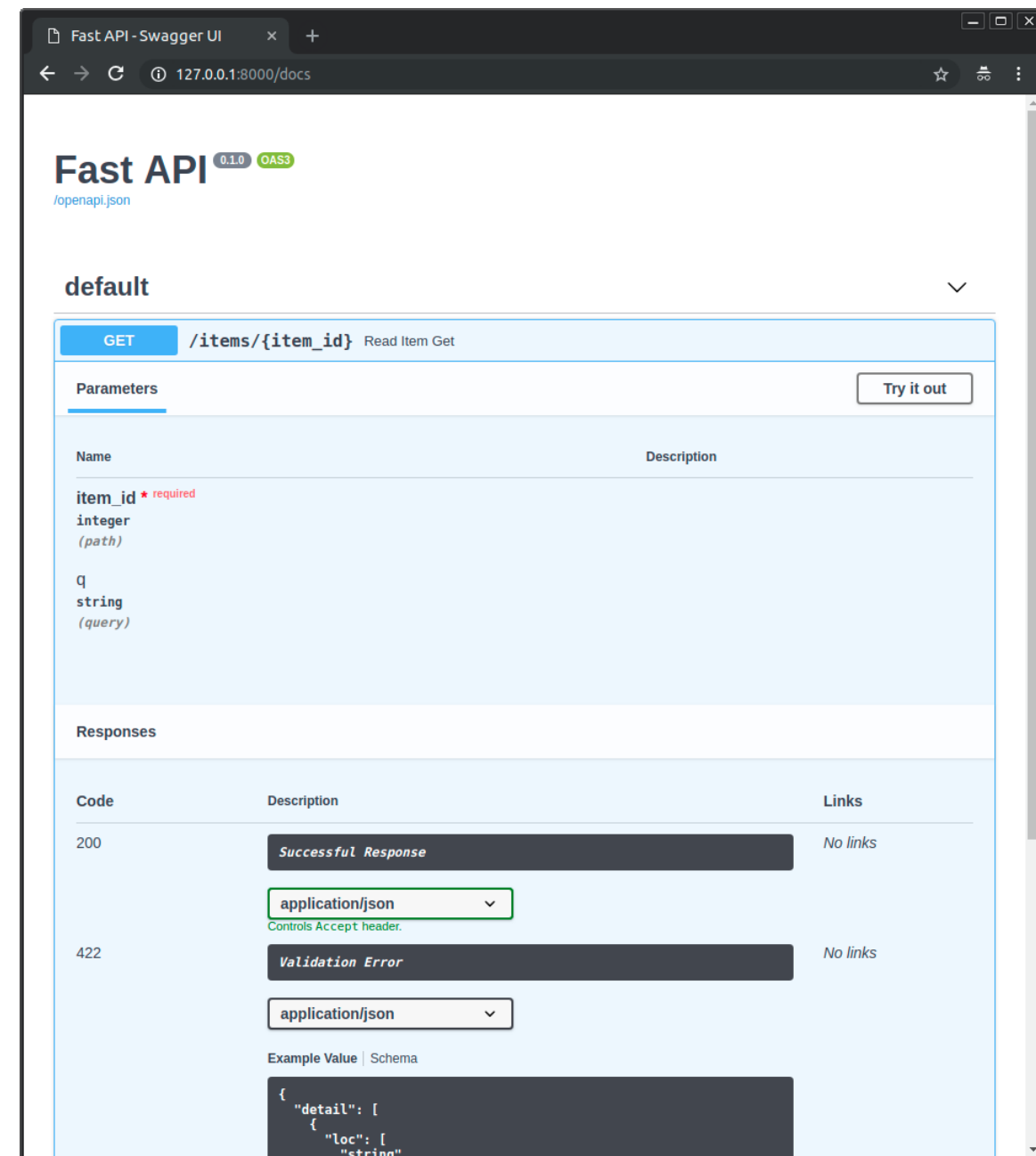
- Запустим сервер в режиме реального времени

```
$ fastapi dev
FastAPI Starting development server 🚀
Searching for package file structure from directories with __init__.py
files
Importing from /home/a.glushenko/project
module 🔄 main.py
code Importing the FastAPI app object from the module with the
following code:
from main import app
app Using import string: main:app
server Server started at http://127.0.0.1:8000
server Documentation at http://127.0.0.1:8000/docs
tip Running in development mode, for production use:
f astapi run
Logs:
INFO Will watch for changes in these directories:
['/home/a.glushenko/project']
INFO Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C
to quit)
INFO Started reloader process [383138] using WatchFiles
INFO Started server process [383153]
INFO Waiting for application startup.
INFO Application startup complete.
```

- Открыв браузер по адресу: <http://127.0.0.1:8000>, мы увидим JSON-ответ вида:

```
{"message": "Hello World"}
```

- Интерактивная документация API. Теперь перейдите по адресу: <http://127.0.0.1:8000/docs>.
- Мы увидим автоматически сгенерированную интерактивную документацию по API (предоставлено Swagger UI):



Практические аспекты

Маршруты (Routes)

APIRouter

Методы HTTP запросов

Модели Pydantic

Ответы (Responses)

Фоновые задачи (Background Tasks)

Работа с файлами

Lifespan(Startup/Shutdowns Events)

Middleware

Маршруты (Routes)

Маршруты определяют, как приложение обрабатывает запросы.

В приведенном ниже примере мы определяем маршрут **GET** `/items/{item_id}`, который принимает параметр `item_id` и возвращает **JSON**-объект с этим параметром.

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/items/{item_id}")
6 def read_item(item_id: int):
7     return {"item_id": item_id}
```

Таким образом, маршруты связывают функции обработчика с путями сервиса. В данном примере метод сервиса `read_item` предоставляет сервис для чтения элемента по идентификатору элемента (**Item Id**).

APIRouter

FastAPI предоставляет удобный инструмент для структурирования вашего приложения, сохраняя при этом всю гибкость.

Чтобы использовать **APIRouter**, вы создаете экземпляр класса **APIRouter** и определяете в нем маршруты.

Затем вы можете легко подключить этот маршрутизатор к основному экземпляру **FastAPI**, используя метод **include_router** класса **FastAPI**.

Использование **APIRouter** делает ваш код более организованным и управляемым, особенно по мере роста сложности и размера вашего **API**.

См. пример на следующем слайде:

APIRouter. Пример

Экземпляры **APIRouter** с именами `v1_router` и `v2_router`.

Например, маршрут `GET / v1_router` возвращает сообщение «**Hello from APIRouter**».

Другой маршрут `GET / v2_router` возвращает то же сообщение, но с датой ответа.

Затем оба маршрутизатора подключаются к маршрутам приложения **FastAPI** с помощью метода `include_router`.

Таким образом, вкратце, **APIRouter** помогает объединять маршруты с похожим поведением в одном месте.

```
1 from fastapi import FastAPI, APIRouter
2
3 app = FastAPI()
4
5 v1_router = APIRouter(prefix="/v1")
6
7 @v1_router.get("/get_message/")
8 def get_message():
9     """
10     Some endpoint.
11     """
12     return {
13         "message": "Hello from APIRouter",
14     }
15
16 v2_router = APIRouter(prefix="/v2")
17
18 @v2_router.get("/get_message/")
19 def get_message_new():
20     """
21     Some endpoint.
22     """
23     return {
24         "message": "Hello from APIRouter",
25         "response_date": "12/04/2024",
26     }
27
28 app.include_router(v1_router, prefix="/api")
29 app.include_router(v2_router, prefix="/api")
```

Методы HTTP запросов

FastAPI поддерживает следующие **HTTP**-методы: **GET**, **POST**, **PUT**, **PATCH**, **OPTIONS** и **HEAD**.

Каждый метод имеет своё назначение и используется для выполнения определённых действий на сервере.

- **GET** — для получения данных,
- **POST** — для создания новых ресурсов,
- **PUT** — для обновления существующих ресурсов,
- **DELETE** — для удаления ресурсов,
- **PATCH** — для частичного обновления ресурсов,
- **OPTIONS** — для получения информации о возможностях сервера,
- **HEAD** — для получения заголовков без тела ответа.

Полный список методов можно найти в RFC 9110, документе, описывающем стандарты.

Модели Pydantic

Pydantic — одна из очень популярных библиотек Python, которая помогает легко определять структуру данных входящего запроса и исходящего ответа.

Pydantic также обеспечивает проверку данных и автоматическую сериализацию между форматами JSON и Python.

Класс **ConfigDict** в **Pydantic** используется для определения дополнительных настроек модели данных.

Атрибут **json_schema_extra** этого класса позволяет добавлять дополнительные метаданные или примеры данных для документации **API**, улучшая читаемость и ясность документации.

Модели Pydantic

В этом примере мы создаём модель `Item`, содержащую поля `name`, `description`, `price` и `tax`.

Поля `name` и `price` являются обязательными, а поля `description` и `tax` — необязательными.

Pydantic автоматически выполняет проверку данных и преобразование типов данных при необходимости.

```
1 from pydantic import BaseModel, ConfigDict
2
3 class Item(BaseModel):
4     name: str
5     description: str = None
6     price: float
7     tax: float = None
8
9     model_config = ConfigDict(
10         json_schema_extra={
11             "example": {
12                 "name": "товар #1",
13                 "description": "описание товара #1",
14                 "price": 17.50,
15                 "tax": 8.5,
16             }
17         }
18     )
```

Ответы (Responses)

FastAPI

предоставляет
удобные методы
для создания
ответов на запросы.

Например,

JSONResponse

для возврата
данных в формате

JSON или **HTMLResponse**

для возврата содержимого в формате **HTML**. Более подробную информацию см. в документации **FastAPI**.

```
1 from fastapi import FastAPI
2 from fastapi.responses import JSONResponse
3 from pydantic import BaseModel
4
5 app = FastAPI()
6
7 @app.get("/items/")
8 def read_item(item_id: int):
9     return JSONResponse(
10         status_code=200,
11         content={"name": "Item 1", "item_id": item_id, "description": "Item description"}
12     )
13
14 class UserIn(BaseModel):
15     username: str
16     password: str
17     email: EmailStr
18     full_name: Union[str, None] = None
19
20 class UserOut(BaseModel):
21     username: str
22     email: EmailStr
23     full_name: Union[str, None] = None
24
25 @app.post("/user/", response_model=UserOut)
26 async def create_user(user: UserIn) -> Any:
27     return user
```

Фоновые задачи (Background Tasks)

Фоновые задачи позволяют выполнять асинхронные операции в фоновом режиме, пока обрабатываются запросы.

Особенности фоновых задач в **FastAPI**:

- **Фоновая обработка:**
 - Фоновые задачи выполняются асинхронно в фоновом режиме, не блокируя ответ на **HTTP**-запрос или основной поток выполнения приложения.
 - Это позволяет приложению продолжать обрабатывать запросы и взаимодействовать с клиентами даже во время выполнения длительных операций.
- **Очередь задач:** возможность **FastAPI** добавлять фоновые задачи в специальную очередь, которая автоматически выполняет их как можно скорее.
- **Взаимодействие с запросами:** фоновые задачи могут получать данные из **HTTP**-запросов и использовать их для выполнения в фоновом режиме.

Lifespan (события запуска и остановки)

Используя параметр **Lifespan**, вы можете определить логику, которая должна выполняться до запуска приложения. Это означает, что этот код должен быть выполнен один раз, до того, как приложение начнет получать запросы.

Аналогично, вы можете определить логику (код), которая должна выполняться при завершении работы приложения. В этом случае этот код будет выполнен один раз после обработки, возможно, большого количества запросов.

Это может быть очень полезно для настройки ресурсов, которые вам необходимо использовать для всего приложения в течение жизненного цикла приложения и/или которые вам необходимо очистить после этого. Например, пул соединений с базой данных или загрузка общей модели машинного обучения.

Lifespan (события запуска и остановки)

```
from contextlib import asynccontextmanager

from fastapi import FastAPI

def fake_answer_to_everything_ml_model(x: float):
    return x * 42

ml_models = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the ML model
    ml_models["answer_to_everything"] = fake_answer_to_everything_ml_model
    yield
    # Clean up the ML models and release the resources
    ml_models.clear()

app = FastAPI(lifespan=lifespan)

@app.get("/predict")
async def predict(x: float):
    result = ml_models["answer_to_everything"](x)
    return {"result": result}
```

Практические аспекты

Правдоподобие, вероятности

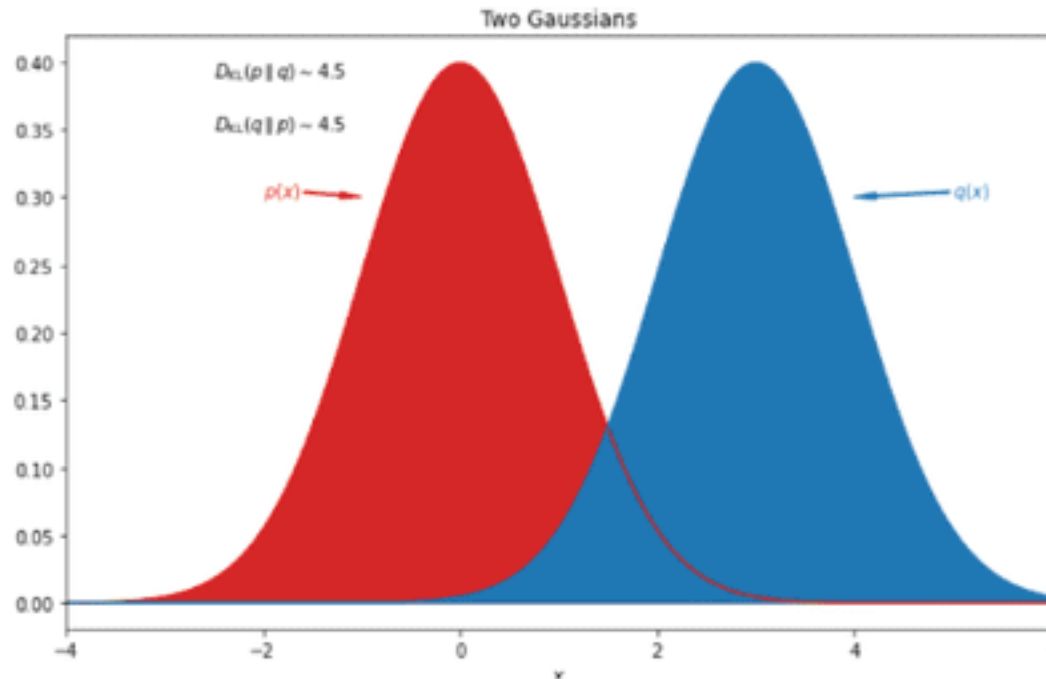
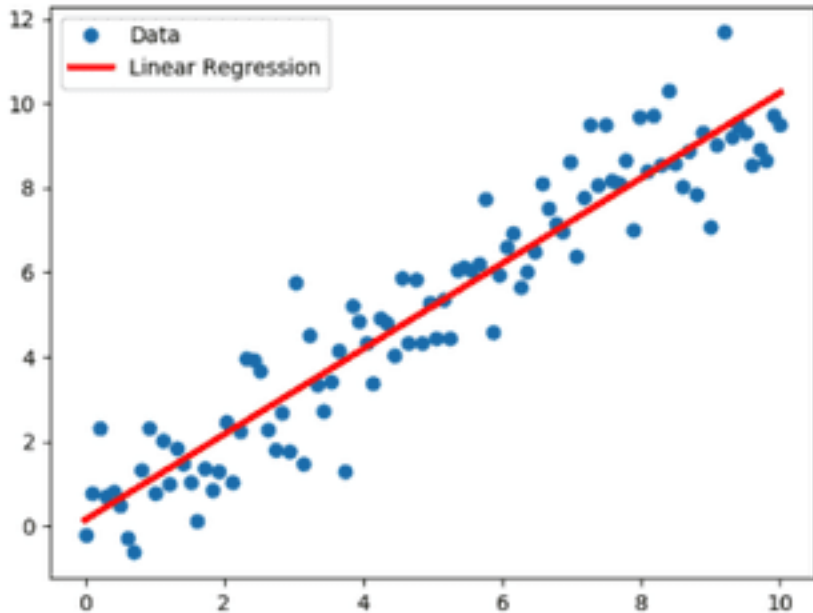
Вероятностный взгляд на регуляризаторы

Формула Байеса

29

Правдоподобие (Likelihood) $P(D|w)$

Это соответствие модели данным. Оно отвечает на вопрос: «Насколько вероятно было получить такие ответы y , если бы параметры нашей модели были равны w ?».



Правдоподобие (Likelihood) $P(D|w)$

В классическом обучении (без регуляризации) мы максимизируем правдоподобие (Maximum Likelihood Estimation — MLE). Для регрессии это эквивалентно минимизации MSE (среднеквадратичной ошибки), а для классификации — минимизации Log-Loss.

Априорная вероятность (Prior) $P(w)$

Это наши убеждения до начала эксперимента. Она отвечает на вопрос: «Насколько вероятны такие веса w сами по себе, безотносительно данных?».

Мы еще не видели данных, но уже говорим: «Скорее всего, коэффициенты модели должны быть около нуля и не должны быть гигантскими».

Априорная вероятность (Prior) $P(w)$

Это и есть регуляризатор.

Если мы считаем, что веса распределены по Гауссу (нормально) — получаем L2.

Если верим, что многие веса должны быть нулевыми (распределение Лапласа) — получаем L1.

Это страховка от переобучения. Мы накладываем ограничения, чтобы модель не стала слишком сложной.

Вероятностный взгляд на регуляризацию

Вероятностный взгляд на регуляризацию позволяет понять, какие априорные ожидания мы закладываем в модель. В основе этого подхода лежит Байесовский вывод.

В обычной линейной регрессии мы ищем веса w , которые максимизируют правдоподобие данных. В регуляризованной — мы ищем веса, которые максимизируют апостериорную вероятность $P(w|X, y)$. По формуле Байеса:

$$P(w|X, y) \propto P(y|X, w) \cdot P(w), \text{ где}$$

$P(y|X, w)$ – правдоподобие, $P(w)$ – апостериорное распределение весов.

L2-регуляризация и распределение Гаусса

Когда мы используем L2, мы предполагаем, что веса распределены по **нормальному закону (Гауссу)** с центром в нуле:

$$P(w) \propto e^{-\frac{\lambda}{2}\|w\|_2^2}$$

Мы верим, что веса должны быть небольшими, но «размазанными». Большие значения крайне маловероятны, но веса редко становятся строго нулевыми.

Логарифмирование этой вероятности даёт нам штраф $\lambda \sum w_i^2$.

L1-регуляризация и распределение Гаусса

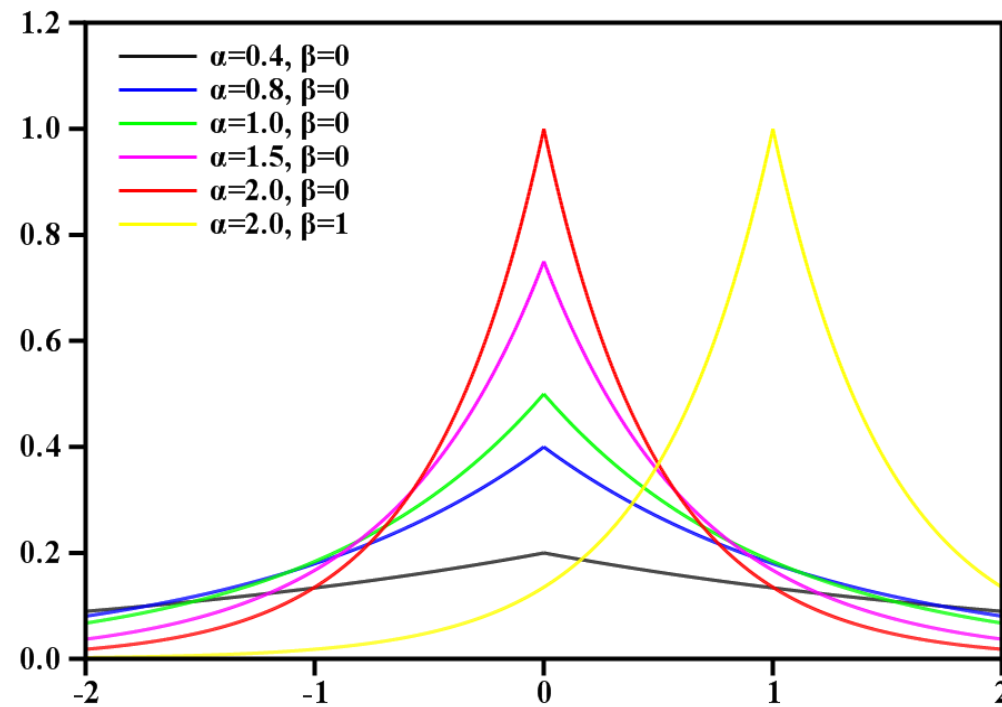
L1 соответствует предположению, что веса подчиняются **распределению Лапласа**:

$$P(w) \propto e^{-\lambda \|w\|_1}$$

У распределения Лапласа острый пик в нуле и «тяжелые хвосты».

Это отражает нашу веру в то, что многие признаки вообще **бесполезны** (их веса должны быть ровно 0), а остальные могут быть довольно значимыми.

Логарифмирование этой вероятности даёт нам штраф $\lambda \sum |w_i|$.



Формула Байеса

В контексте машинного обучения формула Байеса — это способ обновить наши знания о параметрах модели w после того, как мы увидели реальные данные D .

$$P(w|D) = \frac{P(D|w)P(w)}{P(D)}$$

$P(w)$ - априорная вероятность. (Наши предположения о данных до того, как мы открыли датасет)

Пример. Я считаю, что веса должны быть маленькими и распределены по Гауссу.

$P(D|w)$ - правдоподобие. (Вероятность увидеть данные D , если бы мы точно знали, что веса равны w).

Пример. Насколько хорошо прямая с коэффициентами w проходит через точки обучающей выборки?